

Laborator 7 - Algoritmi

1. Implementati solutia **recursivă CUT-ROD** pentru problema tăierii barei de oțel. Implementați abordarea **top-down (procedura MEMOIZED-CUT-ROD)** si abordarea **bottom-up (procedura BOTTOMUP-CUT-ROD)**. Lucrați cu fișiere header.

<pre> MAIN() READ n //lungimea barei READ_PRICES(a,n)//a[i]=price(i) maxvalue := CUT_ROD(a, n) PRINT maxvalue// recursive max value maxvalue := MEMOIZED_CUT_ROD(a, n) PRINT maxvalue// top-down max value maxvalue := BOTTOM_UP_CUT_ROD(a, n) PRINT maxvalue// bottom-up max value END MAIN </pre>	<pre> CUT_ROD(P, N) IF N==0 THEN RETURN 0 END IF q := -∞ FOR i := 1 TO N q := MAX(q, P[i] + CUT_ROD(P, N - i)) END FOR RETURN q END CUT_ROD </pre>
<pre> MAX(A, B) IF A<B THEN RETURN B ELSE RETURN A END IF END MAX READ_PRICES(A, N) FOR i := 1 TO N READ A[i] END FOR END READ_PRICES </pre>	<pre> BOTTOM_UP_CUT_ROD(P, N) Array r[...] r[0] := 0 FOR j := 1 TO N q := -∞ FOR i := 1 TO j q := MAX(q, P[i] + r[j - i]) END FOR r[j] := q END FOR RETURN r[N] END BOTTOM_UP_CUT_ROD </pre>
<pre> MEMOIZED_CUT_ROD(P, N) Array r[...] FOR i := 0 TO N r[i] := -∞ END FOR q := MEMOIZED_CUT_ROD_AUX(P, N, r) //afisare solutii subprobleme PRINT_ARRAY(r, N); RETURN q END MEMOIZED_CUT_ROD </pre>	<pre> MEMOIZED_CUT_ROD_AUX(P, N, R) IF R[N] ≥ 0 THEN RETURN R[N] END IF IF N == 0 THEN q := 0 ELSE q := -∞ FOR i := 1 TO N q := MAX(q,P[i] + +MEMOIZED_CUT_ROD_AUX(P, N-i, R)) END FOR END IF R[N] := q RETURN q END MEMOIZED_CUT_ROD_AUX </pre>

2. Implementati algoritmul determinării modului de punere a parantezelor la inmultirea unui sir de matrici prin abordarea **bottom-up (procedura MATRIX-CHAIN-ORDER)**

<pre> MAIN() Matrix S[..][...] READ n //lungime array=numar //dimensiuni matrici //ex A1*A2*A3 atunci n=4 READ_ARRAY(a, n) MATRIX_CHAIN_ORDER(a, n) PRINT_OPTIMAL_PARENS(1, n-1) END MAIN </pre>	<pre> PRINT_OPTIMAL_PARENS (I, J) IF I == J THEN PRINT "A" I ELSE PRINT "(" PRINT_OPTIMAL_PARENS (I, S[I][J]) PRINT_OPTIMAL_PARENS (S[I][J] + 1, J) PRINT ")" END IF END PRINT_OPTIMAL_PARENS </pre>
<pre> MATRIX_CHAIN_ORDER(P, N) n := N -1 //numar de matrici Matrix m[1..n][1..n] FOR i := 1 TO n m[i][i] := 0 END FOR FOR t := 2 TO n FOR i := 1 TO n - t + 1 j := i + t -1 m[i][j] := +∞ FOR k := i TO j - 1 q := m[i][k] + m[k+1][j] +P[i - 1] x P[k] x P[j] IF q < m[i][j] THEN m[i][j] := q S[i][j] := k END IF END FOR END FOR END FOR END MATRIX_CHAIN_ORDER </pre>	

Obs: Atenție la modul în care se transmite variabila (matricea) S în interiorul subprogramelor implementate in C!!!

Temă. Probleme optionale

1. Pentru abordarea cu programare dinamică a problemei tăierii unei bare de oțel, scrieți o funcție care afișeze soluția optimă obținută.
2. Pentru problema determinării modului de stabilire a parantezelor la inmulțirea unui sir de matrici, scrieți procedura recursivă care determină acest lucru prin **brute-force search** (încercări succesive). Modificați această procedură recursivă astfel incat să obțineți abordarea top-down a programării dinamice.